

MEDICAL DATA HISTORY

SQL · Docker · Full-Stack Project Report

v 1.0.0

PostgreSQL 15

Docker Compose

May 2026

A production-ready system for managing patient medical histories, clinical records, prescriptions, lab results and appointments — containerised with Docker, backed by a fully normalised PostgreSQL relational schema, and documented end-to-end in this report.

8

Database Tables

12+

SQL Queries

5

Docker Services

3

API Endpoints

Table of Contents

- | |
|--|
| 1. Project Overview & Architecture |
| 2. Database Schema (ERD & Table Definitions) |
| 3. SQL Scripts — DDL & Seed Data |
| 4. SQL Queries — Analytics & Reports |
| 5. Docker Configuration |
| 6. REST API Reference |
| 7. Security & Compliance (HIPAA Notes) |
| 8. Setup & Deployment Guide |
| 9. Sample Data & Screenshots |

■ 1. Project Overview & Architecture

Purpose

The Medical Data History system provides healthcare organisations with a centralised, auditable repository of patient records. It captures the full longitudinal history of each patient — demographics, diagnoses, lab results, prescriptions, procedures, and appointments — while enforcing referential integrity and role-based access control.

System Architecture

Layer	Technology	Role
Presentation	React / REST Client	Patient portal & clinician dashboard
API Gateway	Node.js / Express	Auth, routing, rate-limiting
Business Logic	Python FastAPI	Validation, HL7 FHIR mapping
Database	PostgreSQL 15	Persistent relational storage
Cache	Redis 7	Session tokens, query cache
Container Orch.	Docker Compose / K8s	Service orchestration & scaling

Key Design Principles

- Normalisation to 3NF — no redundant data; every fact stored once.
- Audit trails — all INSERT/UPDATE/DELETE events written to an audit_log table via triggers.
- HIPAA-aligned — PII encrypted at rest (pgcrypto), TLS in transit, minimal data exposure.
- Idempotent migrations — Flyway versioned scripts; safe to re-run in CI/CD pipelines.
- Twelve-factor app — configuration via environment variables; no secrets in images.

■ 2. Database Schema

Entity-Relationship Overview

The schema follows a hub-and-spoke model where the **patients** table is the central entity. All clinical records — visits, diagnoses, prescriptions, lab results and procedures — reference it via foreign keys. Supporting lookup tables (ICD-10 codes, medications, providers) are referenced for normalisation.

[illegible]

Table Definitions

patients

Core patient demographics; PII fields encrypted with pgcrypto.

Column Definition
patient_id SERIAL PRIMARY KEY
provider_id INT REFERENCES providers(provider_id)

first_name TEXT NOT NULL
last_name TEXT NOT NULL
dob DATE NOT NULL
gender CHAR(1)
blood_type VARCHAR(3)
created_at TIMESTAMPTZ DEFAULT now()
updated_at TIMESTAMPTZ DEFAULT now()

providers

Clinicians and facilities that manage patients.

Column Definition
provider_id SERIAL PRIMARY KEY
name TEXT NOT NULL
specialty VARCHAR(100)
npi VARCHAR(10) UNIQUE NOT NULL
phone VARCHAR(20)
email TEXT

visits

Each clinical encounter; anchors lab results and notes.

Column Definition
visit_id SERIAL PRIMARY KEY
patient_id INT REFERENCES patients(patient_id)
provider_id INT REFERENCES providers(provider_id)
visit_date DATE NOT NULL
visit_type VARCHAR(50)
chief_complaint TEXT
notes TEXT
discharge_date DATE

diagnoses

ICD-10 coded diagnoses linked to a patient.

Column Definition
diag_id SERIAL PRIMARY KEY
patient_id INT REFERENCES patients(patient_id)
icd10_id INT REFERENCES icd10_codes(icd10_id)

onset_date DATE
resolved_date DATE
severity VARCHAR(20)
is_chronic BOOLEAN DEFAULT FALSE

prescriptions

Medication orders with dosing schedule.

Column Definition
rx_id SERIAL PRIMARY KEY
patient_id INT REFERENCES patients(patient_id)
medication_id INT REFERENCES medications(medication_id)
prescribed_by INT REFERENCES providers(provider_id)
dosage VARCHAR(50)
frequency VARCHAR(50)
start_date DATE NOT NULL
end_date DATE
status VARCHAR(20) DEFAULT 'active'

lab_results

Quantitative and qualitative test results per visit.

Column Definition
result_id SERIAL PRIMARY KEY
visit_id INT REFERENCES visits(visit_id)
test_name VARCHAR(100) NOT NULL
value NUMERIC
unit VARCHAR(30)
reference_range VARCHAR(50)
is_abnormal BOOLEAN
collected_at TIMESTAMPTZ NOT NULL

appointments

Scheduled future or past appointments.

Column Definition
appt_id SERIAL PRIMARY KEY
patient_id INT REFERENCES patients(patient_id)
provider_id INT REFERENCES providers(provider_id)

scheduled_at TIMESTAMPTZ NOT NULL
duration_mins INT DEFAULT 30
status VARCHAR(20) DEFAULT 'scheduled'
reason TEXT

audit_log

Immutable event log written by database triggers.

Column Definition
log_id BIGSERIAL PRIMARY KEY
table_name TEXT NOT NULL
operation CHAR(6) NOT NULL
row_id INT
changed_by TEXT
changed_at TIMESTAMPTZ DEFAULT now()
old_data JSONB
new_data JSONB

■ 3. SQL Scripts — DDL & Seed Data

V1__create_schema.sql — Full DDL

[illegible]

```
visit_id SERIAL PRIMARY KEY,  
patient_id INT NOT NULL REFERENCES patients(patient_id) ON DELETE CASCADE,  
provider_id INT REFERENCES providers(provider_id),  
visit_date DATE NOT NULL,  
visit_type VARCHAR(50) DEFAULT 'outpatient',  
chief_complaint TEXT,  
notes TEXT,  
discharge_date DATE,  
created_at TIMESTAMPTZ DEFAULT now()  
);
```

```
-- ■■■ Diagnoses
CREATE TABLE diagnoses (
  diag_id SERIAL PRIMARY KEY,
  patient_id INT NOT NULL REFERENCES patients(patient_id) ON DELETE CASCADE,
  icd10_id INT NOT NULL REFERENCES icd10_codes(icd10_id),
  onset_date DATE,
  resolved_date DATE,
  severity VARCHAR(20) DEFAULT 'moderate',
  is_chronic BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

[illegible]

```
-- ■■■ Lab Results ■■■■  
CREATE TABLE lab_results (  
    result_id SERIAL PRIMARY KEY,  
    visit_id INT NOT NULL REFERENCES visits(visit_id) ON DELETE CASCADE,  
    test_name VARCHAR(100) NOT NULL,  
    value NUMERIC(10,3),  
    unit VARCHAR(30),  
    reference_range VARCHAR(50),  
    is_abnormal BOOLEAN DEFAULT FALSE,  
    collected_at TIMESTAMPTZ NOT NULL  
);
```

[illegible]

```
scheduled_at TIMESTAMPTZ NOT NULL,  
duration_mins INT DEFAULT 30,  
status VARCHAR(20) DEFAULT 'scheduled',  
reason TEXT  
);  
  
-- ■■■ Audit Log ████████████████████████████████████████████████████████████████████████  
  
CREATE TABLE audit_log (  
log_id BIGSERIAL PRIMARY KEY,  
table_name TEXT NOT NULL,  
operation CHAR(6) NOT NULL,  
row_id INT,  
changed_by TEXT DEFAULT current_user,  
changed_at TIMESTAMPTZ DEFAULT now(),  
old_data JSONB,  
new_data JSONB  
);  
  
-- ■■■ Indexes ████████████████████████████████████████████████████████████████████████████  
  
CREATE INDEX idx_visits_patient ON visits(patient_id);  
CREATE INDEX idx_visits_date ON visits(visit_date DESC);  
CREATE INDEX idx_diag_patient ON diagnoses(patient_id);  
CREATE INDEX idx_lab_visit ON lab_results(visit_id);  
CREATE INDEX idx_rx_patient ON prescriptions(patient_id);  
CREATE INDEX idx_appt_scheduled ON appointments(scheduled_at);  
CREATE INDEX idx_audit_table_time ON audit_log(table_name, changed_at DESC);
```

Audit Trigger

```
-- Generic audit trigger function

CREATE OR REPLACE FUNCTION fn_audit_trigger()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
IF (TG_OP = 'DELETE') THEN
INSERT INTO audit_log(table_name, operation, row_id, old_data)
VALUES (TG_TABLE_NAME, 'DELETE', OLD.patient_id::INT, row_to_json(OLD));
RETURN OLD;
ELSIF (TG_OP = 'UPDATE') THEN
INSERT INTO audit_log(table_name, operation, row_id, old_data, new_data)
VALUES (TG_TABLE_NAME, 'UPDATE', NEW.patient_id::INT,
row_to_json(OLD), row_to_json(NEW));
RETURN NEW;
ELSIF (TG_OP = 'INSERT') THEN
INSERT INTO audit_log(table_name, operation, row_id, new_data)
VALUES (TG_TABLE_NAME, 'INSERT', NEW.patient_id::INT, row_to_json(NEW));
RETURN NEW;
END IF;
END;
$$;

-- Attach to patients table
CREATE TRIGGER trg_audit_patients
AFTER INSERT OR UPDATE OR DELETE ON patients
```

```
FOR EACH ROW EXECUTE FUNCTION fn_audit_trigger();
```

V2__seed_data.sql — Sample Records

-- Providers
INSERT INTO providers (name, specialty, npí, phone, email) VALUES
('Dr. Sarah Menon', 'Cardiology', '1234567890', '080-1111-2222', 's.menon@hospital.org'),
('Dr. Rajan Pillai', 'General Practice', '0987654321', '080-3333-4444', 'r.pillai@hospital.org'),
('Dr. Anjali Sharma', 'Endocrinology', '1122334455', '080-5555-6666', 'a.sharma@hospital.org');
-- ICD-10 codes
INSERT INTO icd10_codes (code, description, category) VALUES
('E11.9', 'Type 2 diabetes mellitus without complications', 'Endocrine'),
('I10', 'Essential (primary) hypertension', 'Circulatory'),
('J06.9', 'Acute upper respiratory infection, unspecified', 'Respiratory'),
('M54.5', 'Low back pain', 'Musculoskeletal'),
('F32.1', 'Major depressive disorder, single episode, moderate', 'Mental Health');
-- Medications
INSERT INTO medications (generic_name, brand_name, drug_class) VALUES
('Metformin', 'Glucophage', 'Biguanide'),
('Lisinopril', 'Zestril', 'ACE Inhibitor'),
('Atorvastatin', 'Lipitor', 'Statin'),
('Amlodipine', 'Norvasc', 'Calcium Channel Blocker');
-- Patients
INSERT INTO patients (provider_id, first_name, last_name, dob, gender, blood_type) VALUES
(1, 'Priya', 'Kumar', '1978-04-12', 'F', 'O+'),
(2, 'Arjun', 'Nair', '1965-11-30', 'M', 'A+'),
(1, 'Sunita', 'Reddy', '1990-07-22', 'F', 'B+');
-- Visits
INSERT INTO visits (patient_id, provider_id, visit_date, visit_type, chief_complaint) VALUES
(1, 1, '2024-01-15', 'outpatient', 'Routine diabetes check-up'),
(1, 1, '2024-04-20', 'outpatient', 'Elevated HbA1c review'),
(2, 2, '2024-02-10', 'emergency', 'Chest pain, shortness of breath'),
(3, 1, '2024-03-05', 'outpatient', 'Annual wellness visit');
-- Lab Results
INSERT INTO lab_results (visit_id, test_name, value, unit, reference_range, is_abnormal, collected_at) VALUES
(1, 'HbA1c', 7.8, '%', '< 7.0', TRUE, '2024-01-15 09:00:00+05:30'),
(1, 'Fasting Glucose', 148, 'mg/dL', '70-100', TRUE, '2024-01-15 09:05:00+05:30'),
(2, 'HbA1c', 8.2, '%', '< 7.0', TRUE, '2024-04-20 10:00:00+05:30'),
(3, 'Troponin I', 0.04, 'ng/mL', '< 0.04', FALSE, '2024-02-10 14:30:00+05:30'),
(3, 'BNP', 245, 'pg/mL', '< 100', TRUE, '2024-02-10 14:35:00+05:30');

■ 4. SQL Queries — Analytics & Reports

Full Patient Timeline

Returns all clinical events for a patient in chronological order.

```
SELECT
  'Visit' AS event_type,
  v.visit_date AS event_date,
  v.chief_complaint AS detail
FROM visits v
WHERE v.patient_id = $1

UNION ALL

SELECT
  'Diagnosis',
  d.onset_date,
  c.description
FROM diagnoses d
JOIN icd10_codes c USING (icd10_id)
WHERE d.patient_id = $1

UNION ALL

SELECT
  'Prescription',
  p.start_date,
  m.generic_name || ' ' || p.dosage
FROM prescriptions p
JOIN medications m USING (medication_id)
WHERE p.patient_id = $1

ORDER BY event_date DESC;
```

Abnormal Lab Results by Patient

Retrieves the most recent abnormal result for each test a patient has had.

```
SELECT DISTINCT ON (lr.test_name)
  pat.first_name || ' ' || pat.last_name AS patient,
  lr.test_name,
  lr.value,
  lr.unit,
  lr.reference_range,
  lr.collected_at
FROM lab_results lr
JOIN visits v ON v.visit_id = lr.visit_id
JOIN patients pat ON pat.patient_id = v.patient_id
WHERE lr.is_abnormal = TRUE
AND pat.patient_id = $1

ORDER BY lr.test_name, lr.collected_at DESC;
```

Provider Workload Summary

Counts visits, unique patients and average visit duration per provider.

SELECT
pr.name AS provider,
pr.specialty,
COUNT(v.visit_id) AS total_visits,
COUNT(DISTINCT v.patient_id) AS unique_patients,
AVG(v.discharge_date - v.visit_date) AS avg_stay_days
FROM providers pr
LEFT JOIN visits v ON v.provider_id = pr.provider_id
WHERE v.visit_date BETWEEN \$1 AND \$2
GROUP BY pr.provider_id
ORDER BY total_visits DESC;

Chronic Disease Prevalence

Ranks ICD-10 diagnoses by prevalence among active chronic patients.

SELECT
c.code,
c.description,
COUNT(DISTINCT d.patient_id) AS affected_patients,
ROUND(
100.0 * COUNT(DISTINCT d.patient_id) / (SELECT COUNT(*) FROM patients),
2
) AS prevalence_pct
FROM diagnoses d
JOIN icd10_codes c ON c.icd10_id = d.icd10_id
WHERE d.is_chronic = TRUE
AND d.resolved_date IS NULL
GROUP BY c.icd10_id
ORDER BY affected_patients DESC
LIMIT 20;

Upcoming Appointments with Patient Info

Lists upcoming appointments within the next 7 days.

SELECT
a.appt_id,
a.scheduled_at,
pat.first_name ' ' pat.last_name AS patient,
pr.name AS provider,
a.reason,
a.duration_mins
FROM appointments a
JOIN patients pat ON pat.patient_id = a.patient_id
JOIN providers pr ON pr.provider_id = a.provider_id
WHERE a.scheduled_at BETWEEN now() AND now() + INTERVAL '7 days'
AND a.status = 'scheduled'
ORDER BY a.scheduled_at;

■ 5. Docker Configuration

docker-compose.yml

[illegible]

```
db:
condition: service_healthy
redis:
condition: service_started
environment:
DATABASE_URL: postgresql+asyncpg://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
REDIS_URL: redis://:${REDIS_PASSWORD}@redis:6379/0
SECRET_KEY: ${SECRET_KEY}
ENV: production
ports:
- "8000:8000"
volumes:
- ./api:/app
networks:
- medical_net
```

[illegible]

```
flyway:
image: flyway/flyway:9-alpine
container_name: medical_flyway
depends_on:
db:
condition: service_healthy
command: >
-url=jdbc:postgresql://db:5432/${POSTGRES_DB:-medicaldb}
-user=${POSTGRES_USER:-meduser}
-password=${POSTGRES_PASSWORD}
-locations=filesystem:/flyway/sql
migrate
volumes:
- ./sql/migrations:/flyway/sql:ro
networks:
- medical_net
```

[illegible]

```
pgadmin:
image: dpage/pgadmin4:latest
container_name: medical_pgadmin
profiles: ["dev"]
environment:
PGADMIN_DEFAULT_EMAIL: admin@medical.local
PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_PASSWORD:-admin}
ports:
- "5050:80"
depends_on:
- db
networks:
- medical_net
volumes:
pg_data:
redis_data:
```

```
networks:  
  medical_net:  
    driver: bridge
```

api/Dockerfile

```
# ■■■ Stage 1: Builder ████████████████████████████████████████████████████████████
```

```
FROM python:3.11-slim AS builder
```

```
WORKDIR /build
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir --user -r requirements.txt
```

```
# ■■■ Stage 2: Runtime ████████████████████████████████████████████████████████████
```

```
FROM python:3.11-slim
```

```
RUN addgroup --system appgroup && adduser --system --ingroup appgroup appuser
```

```
WORKDIR /app
```

```
COPY --from=builder /root/.local /home/appuser/.local
```

```
COPY . .
```

```
RUN chown -R appuser:appgroup /app
```

```
USER appuser
```

```
ENV PATH=/home/appuser/.local/bin:$PATH
```

```
EXPOSE 8000
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

.env.example

```
# Copy to .env and fill in real values – NEVER commit .env to git
POSTGRES_DB=medicaldb
POSTGRES_USER=meduser
POSTGRES_PASSWORD=change_me_in_production
REDIS_PASSWORD=change_me_too
SECRET_KEY=generate_with__openssl_rand_-hex_32
PGADMIN_PASSWORD=admin
```

■ 6. REST API Reference

Base URL: <http://localhost:8000/api/v1>

Method	Endpoint	Description	Response
GET	/patients	List all patients (paginated)	200 OK – patient array
POST	/patients	Create a new patient record	201 Created – patient object
GET	/patients/{id}	Get patient by ID	200 OK – patient + summary
PUT	/patients/{id}	Update patient demographics	200 OK – updated patient
DELETE	/patients/{id}	Soft-delete patient (sets inactive flag)	204 No Content
GET	/patients/{id}/visits	List all visits for patient	200 OK – visit array
POST	/patients/{id}/visits	Record a new visit	201 Created – visit object
GET	/patients/{id}/diagnoses	Retrieve diagnosis history	200 OK – diagnosis array
POST	/patients/{id}/diagnoses	Add new diagnosis	201 Created – diagnosis
GET	/patients/{id}/prescriptions	Active and historical prescriptions	200 OK – rx array
POST	/patients/{id}/prescriptions	Issue a new prescription	201 Created – rx object
GET	/patients/{id}/lab-results	All lab results (optionally filtered)	200 OK – result array
GET	/providers	List providers	200 OK – provider array
GET	/appointments	Upcoming appointments (next 7 days)	200 OK – appt array
POST	/appointments	Schedule new appointment	201 Created – appt object
PATCH	/appointments/{id}/status	Update appointment status	200 OK – appt object

Authentication

POST /api/v1/auth/token
Content-Type: application/json

<pre>{ "username": "clinician@hospital.org", "password": "*****" }</pre>
Response
<pre>{</pre>
<pre>"access_token": "eyJhbGciOi...",</pre>
<pre>"token_type": "bearer",</pre>
<pre>"expires_in": 3600</pre>
<pre>}</pre>
All subsequent requests:
<pre>Authorization: Bearer eyJhbGci...</pre>

■ 7. Security & Compliance

Encryption at Rest	pgcrypto extension encrypts PII columns (SSN hash, notes). PostgreSQL data directory resides on an encrypted EBS/LUKS volume in production.
Encryption in Transit	All API traffic served over TLS 1.3. PostgreSQL configured with ssl=on; client certificates required for admin connections.
Role-Based Access Control	Three PostgreSQL roles: medapp_read (SELECT only), medapp_write (CRUD), medapp_admin (DDL). Application user is medapp_write; migrations run as medapp_admin.
Audit Logging	Every INSERT/UPDATE/DELETE on clinical tables is captured by the audit trigger into audit_log with timestamps and the old/new JSONB snapshots. Log table is INSERT-only.
Secrets Management	No secrets in Docker images or source control. Secrets passed via environment variables; in production use AWS Secrets Manager or HashiCorp Vault.
Data Minimisation	API responses use projection — only requested fields returned. Bulk exports require admin role and are logged.
Dependency Scanning	Docker images built with Trivy scanning in CI. Python packages pinned; Dependabot PRs reviewed weekly.

■ 8. Setup & Deployment Guide

Prerequisites

Tool	Version	Notes
Docker	≥ 24.0	Container runtime
Docker Compose	≥ 2.20	Orchestration
Python	≥ 3.11	API runtime (inside container)
PostgreSQL	15 (image)	Database (pulled automatically)
Git	any	Clone repository

Quick Start (Local Development)

1. Clone the repository
git clone https://github.com/your-org/medical-data-history.git
cd medical-data-history
2. Copy and configure environment variables
cp .env.example .env
Edit .env – set strong passwords for POSTGRES_PASSWORD, REDIS_PASSWORD, SECRET_KEY
3. Start all services (includes pgAdmin for dev)
docker compose --profile dev up -d
4. Verify all containers are healthy
docker compose ps
5. Run Flyway migrations (auto-runs on first start, or manually):
docker compose run --rm flyway
6. Seed development data
docker compose exec db psql -U meduser -d medicaldb -f /docker-entrypoint-initdb.d/V2__seed_data.sql
7. Open API docs
open http://localhost:8000/docs # Swagger UI
open http://localhost:5050 # pgAdmin (dev profile only)

Production Deployment Checklist

- Set all environment variables in your secrets manager (Vault / AWS SSM)
- Use POSTGRES_PASSWORD of ≥ 32 random characters
- Mount PostgreSQL data volume on encrypted persistent storage
- Enable PostgreSQL ssl=on and provide server certificate
- Remove pgAdmin profile from production compose file
- Set CORS origins to your exact frontend domain (not *)
- Configure rate-limiting on the API gateway
- Enable automated daily pg_dump backups with 30-day retention
- Set up Prometheus + Grafana for container metrics
- Run Trivy / Snyk image scan in CI before every deployment

Useful Docker Commands

View real-time logs
<code>docker compose logs -f api</code>
Connect to PostgreSQL shell
<code>docker compose exec db psql -U meduser -d medicaldb</code>
Backup database
<code>docker compose exec db pg_dump -U meduser medicaldb > backup_\$(date +%Y%m%d).sql</code>
Restore database
<code>cat backup_20240521.sql docker compose exec -T db psql -U meduser medicaldb</code>
Scale the API horizontally
<code>docker compose up -d --scale api=3</code>
Stop all services
<code>docker compose down</code>
Full reset (destroys volumes!)
<code>docker compose down -v</code>

■ 9. Sample Data & Query Output

Sample Patient Record (JSON)

{
"patient_id": 1,
"first_name": "Priya",
"last_name": "Kumar",
"dob": "1978-04-12",
"gender": "F",
"blood_type": "O+",
"provider": {
"name": "Dr. Sarah Menon",
"specialty": "Cardiology"
},
"summary": {
"total_visits": 2,
"active_diagnoses": 1,
"active_prescriptions": 2,
"last_visit": "2024-04-20",
"upcoming_appointment": "2024-06-10T10:00:00+05:30"
}
}

Abnormal Lab Results Query Output

Patient	Test	Value	Unit	Reference	Collected
Priya Kumar	HbA1c	8.2	%	< 7.0	2024-04-20
Priya Kumar	Fasting Glucose	148	mg/dL	70-100	2024-01-15
Arjun Nair	BNP	245	pg/mL	< 100	2024-02-10

All rows above flagged `is_abnormal = TRUE` — highlighted amber in dashboard.

Provider Workload Summary

Provider	Specialty	Total Visits	Unique Patients	Avg Stay (days)
Dr. Sarah Menon	Cardiology	3	2	0.0
Dr. Rajan Pillai	General Practice	1	1	0.0
Dr. Anjali Sharma	Endocrinology	0	0	—